ted parameters
- overprovisioned: Using less than the ideal storage percentage (has more reserved space than needed)
- lowwatermarkexceeded: Exceeded the low watermark threshold
- highwatermarkexceeded: Exceeded the high watermark threshold
- criticalwatermarkexceeded: Exceeded the critical watermark threshold

These watermark levels directly influence how storage allocation adjustments are made. Indices in the ready state can both increase their reserved storage when hitting higher watermarks (if node storage is available) or decrease their reservations when overprovisioned.

Storage Reservation Mechanism

The system uses a storage reservation mechanism where:

1. Each index maintains a reservedstoragebytes value representing its allocation
1. The node tracks its total usablestoragebytes and the sum of all index reservations
1. When an index needs more storage, it attempts to claim additional bytes from the node's unclaimed storage
1. Indices in a ready state can both increase and decrease their reservations as needed
1. Indices in initialization states can only increase their reservations until they're fully indexed

This reservation system prevents overcommitment of storage while allowing flexible allocation based on actual needs. If a node approaches its critical watermark, indices may be marked for eviction to reclaim space.

The combination of node-level and index-level watermarks provides a comprehensive approach to storage management, ensuring efficient resource utilization while preventing resource exhaustion at both the node and index levels.

Conclusion

The Zoekt integration significantly improves GitLab's code search capabilities by providing exact match and regular expression search modes. The architecture is designed to be scalable, self-managing, and resilient, with features like node self-registration, automatic sharding, and high availability through replication.

The unified binary approach simplifies deployment and maintenance, while the bidirectional communication between GitLab and Zoekt nodes enables efficient task distribution and status tracking.

Current development efforts focus on enhancing search performance through gRPC-based federated search and improving overall system scalability to support namespaces with more than tens of thousands of projects.

SECTION: engineering/architecture/design-documents/complex_spdx_expression_evaluation/_index.md

<!-- Design Documents often contain forward-looking statements -->
<!-- vale gitlab.FutureTense = NO -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Summary

SPDX license expressions provide a clear, standardized,
and machine-readable way to document the software licenses associated with code,
ensuring compliance with licensing obligations. Standardized expressions help
users manage legal risks, meet licensing requirements, and easily share software
components for reuse. Expressions can be simple, for example an expression
composed of singular SPDX license identifier, or complex, like when more than
one expressions are combined using boolean operators.

Motivation

Projects are increasingly adopting licensing models that make use of complex
license expressions. For example, a project might require both Apache-2.0 and
MIT licenses, or it might let users choose between Apache-2.0 or MIT licenses.
Unfortunately, there's no way to express this with singular license expressions,
so project's that depend on packages that utilize complex license expressions
are left with a dependency list that only mentions an unknown license. This
results in a user experience where it's impossible to quickly view the licensing
options of a dependency, and also presents a problem when a user tries to craft
a license approval policy to fit such an expression. Supporting complex
expressions will improve the development workflow for projects that work with
dependencies that use complex licensing models.

Goals

- Support for simple license identifiers, and the AND and OR boolean
  operators.
- Populate dependency list with license expressions and not just license
  identifiers. For example, a dependency with a Apache-2.0 AND MIT license
  will show up as such in the dependency list.
- Enforce license approval policies on projects that use expressions. For
  example, if a license approval policy only approves of Apache-2.0, and a
  proposed dependency has a Apache-2.0 AND MIT license, the dependency should
  not pass the approval policy. On the other hand, if the approval has acceptance
  for both licenses, it should pass.
- Performance should be acceptable for groups with many projects, and projects
  with many dependencies.

Non-Goals

- The first iteration will not include for the following:
  - WITH operator
  - + operator

- References using AdditionRef-, DocumentRef- or LicenseRef-

Proposal

At a high level, license expressions will propagate from the Package Metadata Database (PMDB). These expressions will surface in the dependency list at both the project and group level, and will also be used when evaluating the approval status of a dependency's license.

Design and implementation details

SPDX license expressions are supported across ecosystems at varying levels. The following table depicts the level at which they're supported by each ecosystem.

Ecosystem	Support level
cargo	fully supported
cocoapods	one or more identifiers - equivalent to conjunctive AND
composer	fully supported
conan	one or more identifiers - equivalent to conjunctive AND
gem	one or more identifiers - equivalent to conjunctive AND
golang	one or more identifiers - equivalent to conjunctive AND
maven	one or more identifiers - equivalent to conjunctive AND
npm	fully supported
pypi	one or more identifiers - equivalent to conjunctive AND
swift	one or more identifiers - equivalent to conjunctive AND

Licensing options

At a high level, a SPDX expression can be compiled down into a set of license combinations. Take the following expression, for example.

text
```
Apache-2.0 AND (MIT OR GLP-3.0)
```

The expression is human readable, deterministic, but it's far from optimized for our use cases. We can improve on this by expanding the expression into the different licensing options it represents. In this case, the options could be represented like so:

json
```
[
   ["Apache-2.0", "MIT"],
   ["Apache-2.0", "GLP-3.0"]
]
```

From here, it's easy to see how a license approval policy can be evaluated against the different licensing options. If an approval policy denies any

of the licenses, the next license option can be tested for denial. The change in logic is minimal, with the only change being an additional outer loop in the violation checking logic.

Evaluation

The solution above is functional, but it must be further improved to meet our performance requirements. As previously mentioned, license scanning must remain performant even for the largest of projects. This solution does not provide that, and we can demonstrate this with the following example.

Say that we scan a project with the following components and licenses, and have an approval policy that we want to evaluate against for policy violations.

```
json
{
   "packageA": [ ["LicenseA", "LicenseB"], ["LicenseA", "LicenseC"]],
   "packageB": [ ["LicenseD", "LicenseE"], ["LicenseD", "LicenseF"]],
   "packageC": [ ["LicenseG", "LicenseH"], ["LicenseG", "LicenseI"]],
}
```

Before evaluating the license approval policies, we'd need to clean up the data, so that we have a singular array of licensing options to evaluate against. The process requires us to combine the options in order to compact the arrays. On first pass, we'd have the following options.

```
json
[
   ["LicenseA", "LicenseB"],
   ["LicenseA", "LicenseC"],
]
```

On second pass, we'd need to combine the existing options with the next set of licensing options, so we'll end up with something like so.

```
json
[
   ["LicenseA", "LicenseB", "LicenseD", "LicenseE"],
   ["LicenseA", "LicenseC", "LicenseD", "LicenseE"],
   ["LicenseA", "LicenseB", "LicenseD", "LicenseF"],
   ["LicenseA", "LicenseC", "LicenseD", "LicenseF"],
]
```

The amount of options has grown considerably, but we're still not done. We must do a third and final pass to incorporate the remaining licensing options for packageC. Doing so results in the following outcome.

json
```
[
    ["LicenseA", "LicenseB", "LicenseD", "LicenseE", "LicenseG", "LicenseH"],
    ["LicenseA", "LicenseC", "LicenseD", "LicenseE", "LicenseG", "LicenseH"],
    ["LicenseA", "LicenseB", "LicenseD", "LicenseF", "LicenseG", "LicenseH"],
    ["LicenseA", "LicenseC", "LicenseD", "LicenseF", "LicenseG", "LicenseH"],
    ["LicenseA", "LicenseB", "LicenseD", "LicenseE", "LicenseG", "LicenseI"],
    ["LicenseA", "LicenseC", "LicenseD", "LicenseE", "LicenseG", "LicenseI"],
    ["LicenseA", "LicenseB", "LicenseD", "LicenseF", "LicenseG", "LicenseI"],
    ["LicenseA", "LicenseC", "LicenseD", "LicenseF", "LicenseG", "LicenseI"],
]
```

From this example, we can see how quickly the search space can become because we're essentially getting the cartesian product of different sets. This combinatorial explosion
requires us to further refine the implementation, so that it remains performant should we be presented with a large amount of dual licensed packages.

To do this, we can make use of bit arrays
to act as a vector of licenses. At a high level the representation would work like this.

- Every bit in the bit array will represent the detection of a license.
- There are 729 known licenses in the SPDX library. If we account for this, future growth, and word alignment, we can use a bit array with 1024 bits.
- We can XOR the bit arrays to detect for approvals or violations. Bitwise operations are fast, and can also be parallelized by the processor.

To demonstrate the implementation, we can do an example using a smaller set of available options. Say that projects have the option of using one or more out of eight licenses. We could represent the licenses like so.

Name	Leftmost Index (MSB)
LicenseA	0
LicenseB	1
LicenseC	2
LicenseD	3
LicenseE	4
LicenseF	5
LicenseG	6
LicenseH	7

Let's say that there was a policy that disallowed LicenseF, and we wanted to evaluate it against the licensing options above. A list approach would have to do a union operation between two lists. Ruby makes this readable and easy to do:

ruby

```
    alldeniedlicenses = (licensesfromreport - licensesfrompolicy).uniq
    comparisonlicenses = joinidsandnames(licenseids, licensenames)
    policydeniedlicensenames = (comparisonlicenses - licensesfrompolicy).uniq
    violateslicensepolicy = policydeniedlicensenames.present?
```

For a singular set of licensing options, it may be acceptable, but using this
for larger sets of data would begin to deteriorate performance. In comparison,
the bit array approach would look like so:

```
ruby
licensesfrompolicy = 0b00000100
licensesfromreport = 0b01010010

def iscompliant?(licensesfrompolicy, licensesfromreport)
  (licensesfromreport & ~licensesfrompolicy) == 0
end

iscompliant?(licensesfrompolicy, licensesfromreport)
=> false
```

Storage

PMDB

The exporter service
for our Package Metadata Database will be in charge of parsing the SPDX
expression, and converting it into an array of bitmasks.

GitLab Rails

The GitLab Rails database will no longer rely on an enum to identify the license used
by a project. Instead it will add an additional column that stores the original
license expression, and its equivalent bitmask representations.

Alternative Solutions

<!--
It might be a good idea to include a list of alternative solutions or paths considered,
although it is not required. Include pros and cons for
each alternative solution/path.

"Do nothing" and its pros and cons could be included in the list too.
-->

SECTION: engineering/architecture/design-documents/compliance-frameworks/_index.md

{{< engineering/design-document-header >}}

Summary

This blueprint serves as living documentation of the technical considerations in the implementation of Compliance Frameworks. This includes functionality in Compliance Frameworks, the Compliance Center and the relationship with Security Policies.

Proposal

Outline how compliance enforcement and visibility will be handled through Compliance Frameworks as an evolution from Compliance Standards Adherence.

Motivation

There are three main parts to compliance posture of a customer Enforcement, Visibility, and Audit History.

Enforcement

Enforcement within GitLab is currently done through Security Policies and Compliance Pipelines. This gives users two very different ways of implementing the same functionality, which has been described as confusing for users. Compliance pipelines also have some inherent technical limitation, see epic.

Visibility

Currently the Standards are hard coded in the Adherence report (renamed to Status report) and separate from Compliance Frameworks. This makes the system inflexible as we look to:

1. Use Compliance Frameworks to include certain projects in the Adherence Report and distinguish which requirements those projects are complaint with.
1. Add a Requirements level to the Adherence report
1. Add more Standards and Controls
1. Allow users to customise Standards
1. Allow users to create their own Standards
1. Allow users to create customisable Controls

Audit History

This is currently achieved through compliance events (Audit events and Violations within MRs).

Background

Deprecate compliance pipelines

We deprecated compliance pipelines in favour of
pipeline execution policy
in GitLab 17.3. This decision was taken to align with the future state of enforcing compliance through security policies.

Scope policies through compliance frameworks

We introduced the capability of scoping security policies allowing us to enforce policies against a specific set of
projects or against projects applied a given set of compliance frameworks. This helped us move towards the
goal of enforcing compliance through frameworks.

Multiple compliance frameworks

Before GitLab 17.3 it was not possible to apply more than one compliance framework to a project. To work towards the
future state of allowing users to customise the adherence dashboard, we created the ability to apply multiple
compliance frameworks in GitLab 17.3.

Goals

1. Provide support need for multiple frameworks
1. Incongruence with compliance pipelines and security policies
1. Use Compliance Frameworks to include certain projects in the Adherence Report and distinguish which requirements those projects are complaint with.
1. Add a Requirements level to the Adherence report
1. Add more Standards and Controls
1. Allow users to customise Standards
1. Allow users to create their own Standards

Non-Goals

1. Compliance events
   1. Audit events
1. Security Policies
   1. This document does not intend to outline how Security Policies work or how Policies use Compliance Frameworks to scope projects
   1. For more information on Security Policies refer this document

Terminology/Glossary

1. Framework
   1. A Compliance Framework is a user-modifiable capability within GitLab to identify projects that have certain compliance requirements or need additional oversight. Compliance Frameworks generally align with established industry compliance frameworks such as SOC2 or ISO 27001.
1. Standard - deprecated in favor of framework
   1. A standard groups together compliance checks in the Adherence report. Compliance standards align with established compliance frameworks/standards such as SOC2 or ISO 27001
1. Adherence
   1. Reports the percentage of a projects compliance posture against a compliance framework. For example if Project A is 50% complaint towards Framework A.
1. Policy
1. Requirement

1. A particular requirement from an industry standard compliance frameworks/standards such as SOC2 or ISO 27001. Usually a statement of intent for a particular part of the compliance framework. These are broken down into specific Controls.
1. Check
   1. A Check is a review of a project's settings, to confirm that it is in a particular position. Checks compose a percentage of a project's compliance posture against a Control.
1. Control
   1. A control is a specific compliance rule that needs to be met to meet a compliance requirement. Enforcement of this is achieved in GitLab through settings, Security Policies or Compliance Pipelines.
1. Violation
   1. A record of an event that when triggered was compared against a Control and found to contravene that control.

Design Details

We will use Sidekiq workers to create controls
and store the adherence configuration in the database as relational data.

See Scalability review document for further details.

Customizable Controls

NOTE: For a more detailed overview, see ADR 003: Custom Controls

We want the ability to create custom requirements so that users don't need to rely only on the exhaustive list of
controls that GitLab supports or would support in the future.

Requirements are composed of a combination of both out-of-the-box and user-defined controls. By building
a normalized and composable data model we avoid special handling for individual controls and can scale both compliance
and violation evaluations uniformally within our relational datastore.

Approach

To allow users to create controls on their own as per their requirements we need to have the following types of
requirements:

1. Internal requirements: Enable users to create logical expressions from an enumerated list of project and namespace computed properties
1. External requirements: Enable users to create requirements that rely on their external services like HTTP servers.

Internal requirements

We will allow users to create logical expressions with all the available project settings.
These expressions form the controls against

which the projects are be evaluated. We store these as a structured JSON in the compliancerequirements table with 'internal'
as the requirementtype.

We will use schema validators for validating the input and store these in the expression column of the
compliancerequirements database table.

The UI will provide dropdowns to choose the field, operator and values. This is created so that the users don't have to write complex JSON expressions on their own.

Each expression is evaluated to a boolean true or false.

External requirements

Users need to be able to create controls where they can configure them to check the state of an external service, as their requirements might not match what GitLab offers by default.

The external HTTP/HTTPS URLs for the user's external services are stored in the compliancerequirementscontrols table with
'external' as the controltype(enum). The same table will also store the shared HMAC secret in the encryptedsecrettoken and encryptedsecrettokeniv columns.

We POST the latest project settings to these external services and expect a HTTP 2xx status as the response.

We provide an API endpoint that can be used to update the status of an external requirement, this would be
similar to setting the status of external status checks.

The shared HMAC secret must be used to sign the request and is also used to check the responses. This
ensures we do not need to use API tokens and complicate role management, while
still ensuring proper authorization.

Since we are only sending project settings for external requirement controls initially,
we expect users to query from our catalog of GitLab APIs to get any
additional information they need to implement the control on their external service.
We can look to expand on the information we send as we receive feature requests for it.

Workflow

1. When evaluating control of a requirement, we send a request to the external service if it has an externalurl defined
   and is of controltype external.
1. After posting we set the corresponding projectcontrolcompliancestatuses entry to state pending and
   allow for a timeout of 30 mins.
1. There will be a separate worker, run with a delay equal to the timeout, checking each control if it

   timed out and is still in state pending, these entries will be defaulted to a fail state.
   (This adds an additional state to what's been mentioned in ADR001))
1. When the external service reports back, we set the status in
   table projectcontrolcompliancestatuses to store the results of the control as the external
   service indicated. ['fail', 'pass']. The external service may update the status of the
   control at any time.

Auditing

Audit events need to be created for the following events in this workflow:

1. Triggering of messages to external service.
1. Non HTTP 2xx statuses encountered when attempting to message external service.
1. Storing replies from external service.
1. Defaulting to a failed state when timeout is reached.
1. Edits done to control (externalurl, secrettoken, etc.)

Application Programmer Interfaces (APIs)

For the external service to be able to post the requirement control results, we need to provide APIs to do so.
This allows external systems to report and query the compliance status of specific project requirements.

API implementations could be implemented along this suggestion.

List all controls

Note: Since each control with externalurl has it's own shared secret,
listing all external controls requires use of a GitLab personal access token (glpat/PAT).

plaintext
```
curl -x GET \
 "https://gitlab.com/api/v4/projects/:id/controlstatuses/" \
  -H 'Authorization: Bearer glpat-XXXXXXXXXXXXXXXXX' \
  -H 'content-type: application/json'
```

Mutation

Note: With appropriate HMAC headers.

graphql
```
mutation UpdateProjectsComplianceControlStatus(
  $id: ID!
  $status: ComplianceState!
```

```
) {
  updateComplianceStatus(
    input: {
      id: $id
      status: $status
    }
  ) {
    complianceStatus {
      id
      status
      updatedAt
    }
    errors
  }
}
```

Database Schema

It was decided to store control expressions in a
separate database table compliancerequirementscontrols.

The compliance requirements would be stored in a separate table with the following schema:

```
mermaid
    classDiagram
    class namespaces {
        id: bigint
        name: text
        path: text
        ...(more columns)
    }
    class projects {
        id: bigint,
        name: text
        path: text
        description: text
        ...(more columns)
    }

    class projectcomplianceframeworksettings {
        id: bigint
        projectid: bigint
        frameworkid: bigint
        ...(more columns)
    }

    class compliancemanagementframeworks {
        id: bigint,
        name: text,
```

```
    description: text,
    ...(more columns)
}

class compliancerequirements {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    namespaceid: bigint
    frameworkid: bigint
    name: text
    description: text
}

class compliancerequirementscontrols {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    namespaceid: bigint
    requirementid: bigint
    name: text
    controltype: smallint
    externalurl: text
    expression: text
    encryptedsecrettoken: bytea
    encryptedsecrettokeniv: bytea
}

class projectcontrolcompliancestatuses {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    projectid: bigint
    namespaceid: bigint
    compliancerequirementid: bigint
    compliancerequirementscontrolid: bigint
    status: smallint
}

class projectcomplianceviolations {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    projectid: bigint
    namespaceid: bigint
    compliancerequirementscontrolsid: bigint
    auditeventid: bigint
    status: smallint
}
```

```
class auditevents {
    id: bigint
    authorid: bigint
    entityid: bigint
    entitytype: string,
    details: text,
    authorname: text,
    entitypath: text,
    targetdetails: text,
    targettype: text,
    targetid: bigint
    ...(more columns)
}

namespaces --> projects  : hasmany

compliancemanagementframeworks <--> projectcompliancefameworksettings : hasmany
projectcompliancefameworksettings <--> projects : manytomany
namespaces --> compliancemanagementframeworks : hasmany
projects --> projectcontrolcompliancestatuses : hasmany
projects --> projectcomplianceviolations : hasmany
compliancemanagementframeworks --> compliancerequirements : hasmany
compliancerequirementscontrols --> projectcontrolcompliancestatuses : hasmany
compliancerequirements --> compliancerequirementscontrols : hasmany
projectcontrolcompliancestatuses <--> auditevents
compliancerequirementscontrols --> projectcomplianceviolations : hasmany
projectcomplianceviolations --> auditevents : hasone
projectcomplianceviolations <-- auditevents : hasmany
```

We plan on dropping the existing projectcompliancestandardsadherence table. We no longer have a standard column
as we don't want to associate requirements directly with a standard, allowing the users to customise
and group requirements as per their need.

Unlike the current implementation we would only store results for the projects that have compliance requirements
configured. Instead of an enum we would store the compliancerequirementid in the projectcontrolcompliancestatuses table and would display these results at the compliance dashboard.

Violations records are stored in the new table projectcomplianceviolations. These violation records are immutable and only new records inserted, unlike the projectcontrolcompliancestatuses table which is updated on status changes. This creates an immutable history of violations against a requirement for a project.

In the next iteration we would also allow importing and exporting the compliance requirement configurations.

Constraints

Feature should be designed with application limits to mitigate abuse, leading to query timeouts and poor user experience.

1. Limit maximum number of compliance frameworks per project: 20 to be increased as needed
1. Limit maximum number of requirements per framework: 50 to be increased as needed
1. Limit maximum number of checks a control expression can have: 5 to be increased as needed
1. Allowlist of project settings and associations that could be used for creating expressions

Compliance framework workflow diagrams

Compliance framework definition

This workflow diagram shows the creation of Compliance Frameworks, Requirements and Controls, and how security policies are associated with Requirements.

```
mermaid
flowchart TD
    A[User creates Compliance Framework] --> B[User adds Requirements to Framework]
    B --> C[User adds Controls in each Requirement]
    C --> D[User chooses one or more Policies for the Requirement]
    D --> F[User applies Framework to Project]

    A -- insert --> compliancemanagementframeworks@{ shape: cyl }
    B -- insert --> compliancerequirements@{ shape: cyl }
    C -- update --> compliancerequirements@{ shape: cyl }
```

Recurring Configuration Status Checks execution flow

This workflow diagram shows the how Compliance Frameworks trigger a configuration status check against a Project.

```
mermaid
flowchart TD
    F[User applies Framework to Project] --> G[Schedule recurring Configuration check sync job]
    G --> H[Get distinct list of actionable Controls in Frameworks applied to Project]
    H --> I[Loop through Controls]

    I --> TYPE{Control Type?}
    TYPE -- Internal --> J{Control has enforcement mechanism? setting/policy}
    TYPE -- External & has externalurl --> EXT[Post message to external service]

    EXT --> PEND[Set control to pending state]
    PEND --> WAIT{Wait max 30 minutes}
    WAIT -->|No reply| FAIL[Default to failed]
    FAIL --> S
    WAIT -->|Got reply| REPLY[Use reply status]
```

```
    FAIL --> Q
    REPLY --> Q
    REPLY -->|Fail| S

    J -- Yes --> M[Check setting/policies configured correctly]
    J -- No --> N[Evaluate Control compliance]

    M --> O[Result: Pass/Fail]
    N --> O
    O --> Q[Upsert result in DB: projectcontrolcompliancestatuses]
    N -- Fail --> S[Insert violation in DB: projectcompliancevialations]@{ shape: cyl }

    Q --> T[Async Configuration check job repeats every 12 hours]
    T --> G
```

Violation triggers execution flow

This workflow diagram shows how violation status checks are triggered and stored.

```
mermaid
flowchart TD
    %% Event-Triggered Violation Check
    F[User applies Framework to Project] --> U[Async Violation check job triggered by audit
event]
    U --> V[Get all Controls in Framework applied to Project]
    V --> W[Loop through Controls]
    W --> X{Audit Event violates a Control?}
    X -- Yes --> Y[Insert violation in DB: projectcompliancevialations]@{ shape: cyl }
    X -- No --> Z[No action needed]
    Y --> AA[Audit Event occurs]
    Z --> AA
    AA --> U
```

For certain controls defined in GitLab there will be a event trigger point. When this event is triggered for a project the violation engine will check whether the project has a compliance framework configured with that requirement controls. If the project does have this configured then the event will be logged as a violation.

For example when a Merge Request is merged the system will trigger a potential violation event. The violations engine will check if there is a control defined for the project which states all Merge Requests requiring 2 approvers, if the Merge Request has less then 2 then a violation is created from the event.

All GitLab defined controls will have an audit event type configured as its trigger point. We will update the audit event type yml file to include a new parameter that will indicate which control it is associated. One audit event may have multiple controls associated with it, such as when an MR is merged.

Audit history

In the above workflows there will be audit events triggered throughout to give a full history of a projects compliance posture. For example audit events will be logged when a project is evaluated against a control and the result of that evaluation. User can then see when the configuration status changed from one state to another in the past. User can then use the audit event reports or streaming audit events to trigger other workflows.

Audit events will be logged when:

- user takes an action
- configuration check result
- violation check result

Decisions

- ~~001: Triggering Checks~~ (changed, see ADR 004)
- 002: Custom Adherence Report
- ~~003: Custom Controls~~ (changed, see ADR 006)
- 004: Use Time-based Triggers for Controls
- 005: Violations Engine
- 006: Storing Controls in a Separate Table
- 007: External Controls
- 008: Policy Relationships

SECTION: engineering/architecture/design-documents/compliance-frameworks/compliance_security_policy_relationship.md

Context

This documents the full relationship between Compliance Frameworks and Security Policies.

- Security Policy Project can be linked with either Project or Namespace (the record in securityorchestrationpolicyconfigurations table is then created, with securitypolicymanagementprojectid used to store information about selected Security Policy Project),
- Policies are defined in the Security Policy Project in the policy.yml file, and they are also represented in the securitypolicies table,
- A single policy can be scoped to multiple Compliance Frameworks (through the complianceframeworksecuritypolicies join table), although you can leave the policy unscoped or scoped to a selected group or project; when the policy is unscoped, it affects all projects/namespaces linked to associated Security Policy Project.
  - For more details on Security Policy scoping refer the docs here https://docs.gitlab.com/ee/user/applicationsecurity/policies/scope
- For a given Compliance Framework, you can define many Requirements (represented in the compliancerequirements table),
- A single Requirement can be associated with multiple Security Policies (through the securitypolicyrequirements join table), and a single Security Policy can be associated with

multiple Requirements as well; the link between Requirement and Security Policy allows user to use selected Security Policy as the enforcement mechanism for selected Requirement

Entity relationship diagram

```
mermaid
erDiagram
    projects ||--o| securityorchestrationpolicyconfigurations : "links with"
    namespaces ||--o| securityorchestrationpolicyconfigurations : "links with"
    securityorchestrationpolicyconfigurations ||--|| projects : "stores policies in"
    securityorchestrationpolicyconfigurations ||--o{ securitypolicies : contains
    securitypolicies ||--|{ complianceframeworksecuritypolicies : "links through"
    complianceframeworksecuritypolicies }|--|| compliancemanagementframeworks : "scopes to"
    compliancemanagementframeworks ||--o{ compliancerequirements : defines
    compliancerequirements ||--o{ securitypolicyrequirements : "associates with"
    securitypolicyrequirements |o--o| securitypolicies : "associates with"

    projects {
        int id PK
        string name
        string path
    }

    namespaces {
        int id PK
        string name
        string path
    }

    compliancemanagementframeworks {
        int id PK
        string name
        string description
    }

    complianceframeworksecuritypolicies {
        int id PK
        int compliancemanagementframeworkid FK
        int securitypolicyid FK
    }

    securitypolicies {
        int id PK
        string name
        string description
        int securityorchestrationpolicyconfigurationid FK
    }

    securitypolicyrequirements {
        int id PK
```

```
        int securitypolicyid FK
        int compliancerequirementid FK
    }

    compliancerequirements {
        int id PK
        int compliancemanagementframeworkid FK
        string description
    }

    securityorchestrationpolicyconfigurations {
        int id PK
        int projectid FK "configuration can be linked either to projectid"
        int namespaceid FK "or namespaceid, but not both"
        int securitypolicymanagementprojectid FK "defines project where we keep policy.yml
file"
    }
```

SECTION: engineering/architecture/design-documents/compliance-frameworks/compliance_standards_adherence_dashboard_mvc.md

Context

We recently released compliance standards adherence dashboard MVC.
In this iteration we introduced the concept of standard and checks. In the initial iteration we started with
GitLab and SOC 2 standards.

GitLab Standard

The GitLab standard consists of three checks:

- Prevent authors as approvers
- Prevent committers as approvers
- At least two approvals

SOC 2 Standard

The SOC 2 standard consists of one check:

- At least one non-author approval

Approach

1. We would create a service class for each of these checks and this class would be invoked by a Sidekiq worker in
the background.
1. These workers are invoked whenever a project is added, or an associated project or group

setting is changed. The
scan is run on that project to update the standards adherence for that project.
1. We planned to store the results of these checks in a database table with the following schema:

```
mermaid
    classDiagram
class namespaces {
    id: bigint
    name: text
    path: text
    ...(more columns)
}
class projects {
    id: bigint,
    name: text
    path: text
    description: text
    ...(more columns)
}

class projectcompliancestandardsadherence {
    id: bigint
    createdat: timestamp
    updatedat: timestamp
    projectid: bigint
    namespaceid: bigint
    checkname: smallint
    status: smallint
    standard: smallint
}

namespaces --> projects : hasmany
namespaces <-- projects : belongsto
projects --> projectcompliancestandardsadherence : hasmany
projects <-- projectcompliancestandardsadherence : belongsto
```

1. checkname is Enum and stores the names of the checks. Eg: "preventapprovalbymergerequestauthor", "preventapprovalbymergerequestcommitters", "atleasttwoapprovals", etc.
1. standard column stores the name of the standard to which the check belongs to, like SOC 2, GitLab, etc.

Conclusion

We received good feedbacks from the users after the MVC was released, however, some users also expressed their concerns
around the checks being rigid and the inability to configure them as per their requirement. For example: Some of the

users didn't have a requirement to get their merge requests approved by two users. They want the ability to change the
count of required approvers to 1 for their projects. We plan to work on these in the next iteration and have created an
architectural decision record for custom adherence report.

SECTION: engineering/architecture/design-documents/compliance-frameworks/scalability_review.md

Context

In https://gitlab.com/groups/gitlab-org/-/epics/13295 we are delivering a new experience where customer can configure their own compliance requirements through compliance frameworks and then report and enforce those requirements through the adherence report and security policies.

To accommodate large-scale implementation of compliance frameworks (1000+ projects), we must prioritize scalability and ensure the feature remains performant even under heavy workloads.

Due to the configurability and importance of these features it is critical to ensure a smooth experience and adaptability of the features.

Proposal

- x] Review and test the current DB ERD and [review how the DB queries will scale for large groups (1000+ projects)
- [x] Review the current ADRs and technical review.
- x] [Review the effort it would take to add new controls

Feature Overview

- A Compliance Framework belongs to a Namespace
  - A Framework belongs to many Projects within a Namespace
  - A Framework has many Requirements
    - A Requirement is composed of a Type (internal or external) and Control Expression
      - A Control Expression defines the relevant setting or configuration to be checked

See Design Details for detailed ERD

Compliance evaluation flow

Given a namespace with two frameworks, from a high-level evaluation of a framework is performed using the following steps:

1. A namespace setting is modified (worst case cascading and applicable to all Ultimate projects within said namespace)
1. Given FrameworkA, we must evaluate all projects associated to FrameworkA within the namespace
1. IDs of all associated projects are fetched and enqueued as batched background jobs to check

each project's compliance status against FrameworkA's controls
1. Given ProjectA, within the background Sidekiq worker we fetch the project and associate data by ID.
    1. We iterate over FrameworkA's requirements to evaluate controls
    1. Entries within projectcomplianceconfigurationstatus are upserted with the results of the control evaluation
1. Given FrameworkB, we must evaluate all projects associated to FrameworkB within the namespace
1. IDs of all associated projects are fetched and enqueued as batched background jobs to check each project's compliance status against FrameworkB's controls
1. Given ProjectA, within the background Sidekiq worker we fetch the project and associate data by ID.
    1. We iterate over FrameworkB's requirements to evaluate controls
    1. Entries within projectcomplianceconfigurationstatus are upserted with the results of the control evaluation
1. Given ProjectB, within the background Sidekiq worker we fetch the project and associate data by ID.
    1. We iterate over FrameworkB's requirements to evaluate controls
    1. Entries within projectcomplianceconfigurationstatus are upserted with the results of the control evaluation

Query plan validation

On modification of a project setting, using feature branch.

1. Retrieval of frameworks that have compliance requirements

rb
frameworks = ComplianceManagementFramework.joins(:compliancerequirements).distinct.order(:id)

<details><summary>click to expand query plan</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32134/commands/99394

sql
SELECT DISTINCT
    "compliancemanagementframeworks".
FROM
    "compliancemanagementframeworks"
    INNER JOIN "compliancerequirements" ON "compliancerequirements"."frameworkid" = "compliancemanagementframeworks"."id"
ORDER BY
    "compliancemanagementframeworks"."id" ASC

</details>

2. Retrieval of project ID batches by associated framework

rb
```
frameworks.each do |framework|
  framework.projects.eachbatch(of: 100) do |project|
    ProjectComplianceEvaluatorWorker.performasync(framework.id, project.pluckprimarykey)
  end
end
```

<details><summary>click to expand query plan</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99421

sql
```
select
    "projects"."id"
from
    projects
    join projectcompliancframeworksettings on projectcompliancframeworksettings.projectid =
projects.id
where
    projectcompliancframeworksettings.frameworkid = 1019907 OFFSET 100
LIMIT 1;
```

</details>

3. Retrieval of projects and associated tables via background worker

Within a batch background worker we must fetch a batch of projects for a given framework. We also preload all relevant associations to perform evaluations against the framework's controls.

This requires a larger initial set of queries that may include attributes we are not utilizing but prevents fan-out of redundant queries within evaluation of each Requirement's Controls.

rb
```
Project.includes(
    :cicdsettings,
    :projectfeature,
    :projectsetting,
    :protectedbranches,
    :securitysetting
).where(id: projectids)
```

<details><summary>Fetching projects by ID</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99422

sql

```
SELECT
    "projects"."id",
    "projects"."name",
    "projects"."path",
    "projects"."description",
    "projects"."createdat",
    "projects"."updatedat",
    "projects"."creatorid",
    "projects"."namespaceid",
    "projects"."lastactivityat",
    "projects"."importurl",
    "projects"."visibilitylevel",
    "projects"."archived",
    "projects"."avatar",
    "projects"."mergerequeststemplate",
    "projects"."starcount",
    "projects"."mergerequestsrebaseenabled",
    "projects"."importtype",
    "projects"."importsource",
    "projects"."approvalsbeforemerge",
    "projects"."resetapprovalsonpush",
    "projects"."mergerequestsffonlyenabled",
    "projects"."issuestemplate",
    "projects"."mirror",
    "projects"."mirrorlastupdateat",
    "projects"."mirrorlastsuccessfulupdateat",
    "projects"."mirroruserid",
    "projects"."sharedrunnersenabled",
    "projects"."runnerstoken",
    "projects"."buildallowgitfetch",
    "projects"."buildtimeout",
    "projects"."mirrortriggerbuilds",
    "projects"."pendingdelete",
    "projects"."publicbuilds",
    "projects"."lastrepositorycheckfailed",
    "projects"."lastrepositorycheckat",
    "projects"."onlyallowmergeifpipelinesucceeds",
    "projects"."hasexternalissuetracker",
    "projects"."repositorystorage",
    "projects"."repositoryreadonly",
    "projects"."requestaccessenabled",
    "projects"."hasexternalwiki",
    "projects"."ciconfigpath",
    "projects"."lfsenabled",
    "projects"."descriptionhtml",
    "projects"."onlyallowmergeifalldiscussionsareresolved",
    "projects"."repositorysizelimit",
    "projects"."printingmergerequestlinkenabled",
    "projects"."autocancelpendingpipelines",
    "projects"."servicedeskenabled",
```

```
    "projects"."cachedmarkdownversion",
    "projects"."deleteerror",
    "projects"."lastrepositoryupdatedat",
    "projects"."disableoverridingapproverspermergerequest",
    "projects"."storageversion",
    "projects"."resolveoutdateddiffdiscussions",
    "projects"."remotemirroravailableoverridden",
    "projects"."onlymirrorprotectedbranches",
    "projects"."pullmirroravailableoverridden",
    "projects"."jobscacheindex",
    "projects"."externalauthorizationclassificationlabel",
    "projects"."mirroroverwritesdivergedbranches",
    "projects"."pageshttpsonly",
    "projects"."externalwebhooktoken",
    "projects"."packagesenabled",
    "projects"."mergerequestsauthorapproval",
    "projects"."poolrepositoryid",
    "projects"."runnerstokenencrypted",
    "projects"."bfgobjectmap",
    "projects"."detectedrepositorylanguages",
    "projects"."mergerequestsdisablecommittersapproval",
    "projects"."requirepasswordtoapprove",
    "projects"."maxpagessize",
    "projects"."maxartifactssize",
    "projects"."pullmirrorbranchprefix",
    "projects"."removesourcebranchaftermerge",
    "projects"."markedfordeletionat",
    "projects"."markedfordeletionbyuserid",
    "projects"."autoclosereferencedissues",
    "projects"."suggestioncommitmessage",
    "projects"."projectnamespaceid",
    "projects"."hidden",
    "projects"."organizationid"
FROM
    "projects"
WHERE
    "projects"."id" IN (13083, 13764, 14022, 14288, 14289, 16648, 19776, 20085, 20086, 20699,
23081, 27470, 27726, 29286, 36743, 72724, 74823, 83282, 98024, 116212, 140724, 143237, 145205,
150440, 227582, 250324, 250833, 278964, 280425, 375711, 387896, 413007, 430285, 443787, 444821,
455030, 480929, 554859, 593728, 629054, 629060, 684698, 730448, 734943, 747741, 766015, 818896,
876090, 887372, 928825, 931715, 998792, 1075790, 1120019, 1209837, 1265999, 1329047, 1379171,
1441932, 1470839, 1533158, 1777822, 1794617, 1911766, 1990920, 2009901, 2127625, 2317465,
2337675, 2347063, 2383700, 2651596, 2670515, 2694799, 2725567, 2890326, 2953390, 3010986,
3010998, 3094319, 3101096, 3305972, 3466815, 3588247, 3605985, 3631141, 3651684, 3662568,
3662668, 3674569, 3698388, 3871132, 3871556, 3885956, 3885980, 3933206, 3933372, 3991945,
4108541, 4121724)


</details>
```

<details><summary>Fetching projectcicdsettings by project ID</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99423

sql
SELECT "projectcicdsettings". FROM "projectcicdsettings" WHERE "projectcicdsettings"."projectid" IN (13083, 13764, 14022, 14288, 14289, 16648, 19776, 20085, 20086, 20699, 23081, 27470, 27726, 29286, 36743, 72724, 74823, 83282, 98024, 116212, 140724, 143237, 145205, 150440, 227582, 250324, 250833, 278964, 280425, 375711, 387896, 413007, 430285, 443787, 444821, 455030, 480929, 554859, 593728, 629054, 629060, 684698, 730448, 734943, 747741, 766015, 818896, 876090, 887372, 928825, 931715, 998792, 1075790, 1120019, 1209837, 1265999, 1329047, 1379171, 1441932, 1470839, 1533158, 1777822, 1794617, 1911766, 1990920, 2009901, 2127625, 2317465, 2337675, 2347063, 2383700, 2651596, 2670515, 2694799, 2725567, 2890326, 2953390, 3010986, 3010998, 3094319, 3101096, 3305972, 3466815, 3588247, 3605985, 3631141, 3651684, 3662568, 3662668, 3674569, 3698388, 3871132, 3871556, 3885956, 3885980, 3933206, 3933372, 3991945, 4108541, 4121724)

</details>

<details><summary>Fetching projectfeatures by project ID</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99424

sql
SELECT "projectfeatures". FROM "projectfeatures" WHERE "projectfeatures"."projectid" IN (13083, 13764, 14022, 14288, 14289, 16648, 19776, 20085, 20086, 20699, 23081, 27470, 27726, 29286, 36743, 72724, 74823, 83282, 98024, 116212, 140724, 143237, 145205, 150440, 227582, 250324, 250833, 278964, 280425, 375711, 387896, 413007, 430285, 443787, 444821, 455030, 480929, 554859, 593728, 629054, 629060, 684698, 730448, 734943, 747741, 766015, 818896, 876090, 887372, 928825, 931715, 998792, 1075790, 1120019, 1209837, 1265999, 1329047, 1379171, 1441932, 1470839, 1533158, 1777822, 1794617, 1911766, 1990920, 2009901, 2127625, 2317465, 2337675, 2347063, 2383700, 2651596, 2670515, 2694799, 2725567, 2890326, 2953390, 3010986, 3010998, 3094319, 3101096, 3305972, 3466815, 3588247, 3605985, 3631141, 3651684, 3662568, 3662668, 3674569, 3698388, 3871132, 3871556, 3885956, 3885980, 3933206, 3933372, 3991945, 4108541, 4121724)

</details>

<details><summary>Fetching projectsecuritysettings by project ID</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99425

sql
SELECT "projectsecuritysettings". FROM "projectsecuritysettings" WHERE "projectsecuritysettings"."projectid" IN (13083, 13764, 14022, 14288, 14289, 16648, 19776, 20085, 20086, 20699, 23081, 27470, 27726, 29286, 36743, 72724, 74823, 83282, 98024, 116212, 140724, 143237, 145205, 150440, 227582, 250324, 250833, 278964, 280425, 375711, 387896, 413007, 430285, 443787, 444821, 455030, 480929, 554859, 593728, 629054, 629060, 684698, 730448, 734943, 747741, 766015, 818896, 876090, 887372, 928825, 931715, 998792, 1075790, 1120019, 1209837,

1265999, 1329047, 1379171, 1441932, 1470839, 1533158, 1777822, 1794617, 1911766, 1990920, 2009901, 2127625, 2317465, 2337675, 2347063, 2383700, 2651596, 2670515, 2694799, 2725567, 2890326, 2953390, 3010986, 3010998, 3094319, 3101096, 3305972, 3466815, 3588247, 3605985, 3631141, 3651684, 3662568, 3662668, 3674569, 3698388, 3871132, 3871556, 3885956, 3885980, 3933206, 3933372, 3991945, 4108541, 4121724)

</details>

<details><summary>Fetching projectsettings by project ID</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99426

sql
EXPLAIN SELECT "projectsettings"."projectid", "projectsettings"."createdat", "projectsettings"."updatedat", "projectsettings"."pushruleid", "projectsettings"."showdefaultawardemojis", "projectsettings"."allowmergeonskippedpipeline", "projectsettings"."squashoption", "projectsettings"."hasconfluence", "projectsettings"."hasvulnerabilities", "projectsettings"."preventmergewithoutjiraissue", "projectsettings"."cveidrequestenabled", "projectsettings"."mrdefaulttargetself", "projectsettings"."previousdefaultbranch", "projectsettings"."warnaboutpotentiallyunwantedcharacters", "projectsettings"."mergecommittemplate", "projectsettings"."hasshimo", "projectsettings"."squashcommittemplate", "projectsettings"."legacyopensourcelicenseavailable", "projectsettings"."targetplatforms", "projectsettings"."enforceauthchecksonuploads", "projectsettings"."selectivecodeownerremovals", "projectsettings"."showdiffpreviewinemail", "projectsettings"."suggestedreviewersenabled", "projectsettings"."onlyallowmergeifallstatuscheckspassed", "projectsettings"."issuebranchtemplate", "projectsettings"."mirrorbranchregex", "projectsettings"."allowpipelinetriggerapprovedeployment", "projectsettings"."emailsenabled", "projectsettings"."pagesuniquedomainenabled", "projectsettings"."pagesuniquedomain", "projectsettings"."runnerregistrationenabled", "projectsettings"."productanalyticsinstrumentationkey", "projectsettings"."productanalyticsdatacollectorhost", "projectsettings"."cubeapibaseurl", "projectsettings"."encryptedcubeapikey", "projectsettings"."encryptedcubeapikeyiv", "projectsettings"."encryptedproductanalyticsconfiguratorconnectionstring", "projectsettings"."encryptedproductanalyticsconfiguratorconnectionstringiv", "projectsettings"."pagesmultipleversionsenabled", "projectsettings"."allowmergewithoutpipeline", "projectsettings"."duofeaturesenabled", "projectsettings"."requirereauthenticationtoapprove", "projectsettings"."observabilityalertsenabled", "projectsettings"."spprepositorypipelineaccess" FROM "projectsettings" WHERE "projectsettings"."projectid" IN (13083, 13764, 14022, 14288, 14289, 16648, 19776, 20085, 20086, 20699, 23081, 27470, 27726, 29286, 36743, 72724, 74823, 83282, 98024, 116212, 140724, 143237, 145205, 150440, 227582, 250324, 250833, 278964, 280425, 375711, 387896, 413007, 430285, 443787, 444821, 455030, 480929, 554859, 593728, 629054, 629060, 684698, 730448, 734943, 747741, 766015, 818896, 876090, 887372, 928825, 931715, 998792, 1075790, 1120019, 1209837, 1265999, 1329047, 1379171, 1441932, 1470839, 1533158, 1777822, 1794617, 1911766, 1990920, 2009901, 2127625, 2317465, 2337675, 2347063, 2383700, 2651596, 2670515, 2694799, 2725567, 2890326, 2953390, 3010986, 3010998, 3094319, 3101096, 3305972, 3466815, 3588247, 3605985, 3631141, 3651684, 3662568, 3662668, 3674569, 3698388, 3871132,

3871556, 3885956, 3885980, 3933206, 3933372, 3991945, 4108541, 4121724)

</details>

<details><summary>Fetching approvalrules by projectid</summary>

For atleasttwoapprovals or any other control related to approval rule

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32134/commands/99302

```
sql
SELECT
   SUM(approvalsrequired)
FROM
   "approvalprojectrules"
WHERE
   "approvalprojectrules"."projectid" = 1
LIMIT 1
```

</details>

<details><summary>Fetching protectedbranches by projectid</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32134/commands/99303

```
sql
SELECT
   "protectedbranches".
FROM (
   SELECT
      "protectedbranches".
   FROM
      "protectedbranches"
   WHERE
      "protectedbranches"."projectid" = 1) protectedbranches
```

</details>

<details><summary>Fetching namespace by namespaceid</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32134/commands/99304

```
sql
SELECT
   "namespaces"."id",
   "namespaces"."name",
   "namespaces"."path",
```

```
    "namespaces"."ownerid",
    "namespaces"."createdat",
    "namespaces"."updatedat",
    "namespaces"."type",
    "namespaces"."description",
    "namespaces"."avatar",
    "namespaces"."membershiplock",
    "namespaces"."sharewithgrouplock",
    "namespaces"."visibilitylevel",
    "namespaces"."requestaccessenabled",
    "namespaces"."ldapsyncstatus",
    "namespaces"."ldapsyncerror",
    "namespaces"."ldapsynclastupdateat",
    "namespaces"."ldapsynclastsuccessfulupdateat",
    "namespaces"."ldapsynclastsyncat",
    "namespaces"."descriptionhtml",
    "namespaces"."lfsenabled",
    "namespaces"."parentid",
    "namespaces"."sharedrunnersminuteslimit",
    "namespaces"."repositorysizelimit",
    "namespaces"."requiretwofactorauthentication",
    "namespaces"."twofactorgraceperiod",
    "namespaces"."cachedmarkdownversion",
    "namespaces"."projectcreationlevel",
    "namespaces"."runnerstoken",
    "namespaces"."filetemplateprojectid",
    "namespaces"."samldiscoverytoken",
    "namespaces"."runnerstokenencrypted",
    "namespaces"."customprojecttemplatesgroupid",
    "namespaces"."autodevopsenabled",
    "namespaces"."extrasharedrunnersminuteslimit",
    "namespaces"."lastciminutesnotificationat",
    "namespaces"."lastciminutesusagenotificationlevel",
    "namespaces"."subgroupcreationlevel",
    "namespaces"."maxpagessize",
    "namespaces"."maxartifactssize",
    "namespaces"."mentionsdisabled",
    "namespaces"."defaultbranchprotection",
    "namespaces"."maxpersonalaccesstokenlifetime",
    "namespaces"."pushruleid",
    "namespaces"."sharedrunnersenabled",
    "namespaces"."allowdescendantsoverridedisabledsharedrunners",
    "namespaces"."traversalids",
    "namespaces"."organizationid"
FROM
    "namespaces"
WHERE
    "namespaces"."type" = 'Group'
    AND "namespaces"."id" = 22
LIMIT 1
```

</details>

4. Retrieval of Compliance Requirements for given framework

<details><summary>click to expand query plan</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32139/commands/99427

sql
```
SELECT
   "compliancerequirements".
FROM
   "compliancerequirements"
WHERE
   "compliancerequirements"."frameworkid" = 1020460
```

</details>

5. For each requirement, evaluate controlexpressions against project

Given all required associations are preloaded initially, all control expression evaluation happens in-memory requiring no lookups

rb
```
projects.each do |project|
  framework.compliancerequirements.each do |req|
    ComplianceManagement::ComplianceRequirement::QueryEvaluator.new(Gitlab::Json.parse(req.cont
rolexpression), project).evaluate
  end
end
```

Query plan: none

6. Persistence of project compliance status

After evaluating the controlexpression to true/false we can insert the result in projectcomplianceconfigurationstatus which produces the following SQL

<details><summary>click to expand query plan</summary>

https://console.postgres.ai/gitlab/gitlab-production-main/sessions/32134/commands/99315

sql
```
INSERT INTO "projectcomplianceconfigurationstatus" ("createdat", "updatedat", "projectid",
"namespaceid", "compliancerequirementid", "status")
   VALUES ('2024-09-27 20:28:14.097687', '2024-09-27 20:28:14.097687', 20, 31, 15, 1)
RETURNING
```

"id"

</details>

7. Rendering of Project Compliance Configuration Status dashboard

We need to fetch the rows from projectcomplianceconfigurationstatus table to display on the dashboard, these queries would be very similar to the existing query for fetching records from projectcompliancestandardsadherence.

<details><summary>click to expand query plan</summary>

</details>

Trigger conditions

There are two paths by which a framework evaluation could be enqueued: via setting modification or via time.

When a namespace setting is modified, for example, jobs may be enqueued to re-evaluate the namespace's frameworks against their applicable projects. Similarly, when a project setting is modified, a job is enqueued to re-evaluate the project against its applicable frameworks.

The main disadvantage with an event-based trigger is that we have to maintain an extensive map of possible trigger points. This can be difficult to scale, requires special code for any newly-added controls, and prone to breakage or inadvertent removal.

Additionally, some controls may be impossible to tie to event-based reconciliation. In such cases, a control expression must be evaluated on a recurring cadence; i.e. checking to ensure vulnerabilities are triaged within an anticipated SLO. In these cases the framework must trigger a re-evaluation frequently.

For this reason, the decision was made to use a cron-based approach to perform recurring project compliance evaluations while triggering one-time executions when frameworks are first applied to projects.

See ADR 004 for more on this decision.

Implementation of new controls

Custom controls are composed of two components which must both be implemented to add new controls:

1. Condition Name for use within control expressions
1. Condition Evaluator

To implement new controls we must define a condition name that generally maps to a GitLab-native entity (i.e. a specific project setting). This control must be defined within the compliancerequirementexpression JSON schema as a valid expression enumeration.

To implement a condition evaluator, code must be added to the query evaluator for determining whether a project is compliant with the condition. This requires a custom implementation per control category to perform a boolean evaluation of a specific compliance requirement.

Control category

New control additions must consider the required data to perform an evaluation. There is number of prerequisite tables that are automatically loaded for evalution (see example in Retrieval of projects and associated tables via background worker) that generally correspond to base requirements for each group of query evaluators. When requiring a new control against securitysettings, for example, such a base class as ComplianceManagement::SecuritySettingConditionEvaluator can be extended to ensure all data is loaded and evaluated uniformally.

Example

Example addition of an additional project settings field with a new condition schema and evaluator

patch
```
diff --git a/ee/app/validators/jsonschemas/compliancerequirementexpression.json
b/ee/app/validators/jsonschemas/compliancerequirementexpression.json
index 2716c9e702b2..f8f454368a76 100644
--- a/ee/app/validators/jsonschemas/compliancerequirementexpression.json
+++ b/ee/app/validators/jsonschemas/compliancerequirementexpression.json
@@ -56,6 +56,7 @@
            "projectname",
            "projectvisibility",
            "removeapprovalswhennewcommitsareadded",
+           "projectdisableoverridingapproverspermergerequest",
            "minimumapprovalsrequired"
          ]
        },
@@ -120,6 +121,7 @@
              "enum": [
                "defaultbranchprotected",
                "removeapprovalswhennewcommitsareadded",
+               "projectdisableoverridingapproverspermergerequest",
                "atleasttwoapprovals",
                "preventapprovalbymergerequestcommitters",
                "preventapprovalbymergerequestauthor"
diff --git a/ee/lib/compliancemanagement/compliancerequirement/queryevaluator.rb
b/ee/lib/compliancemanagement/compliancerequirement/queryevaluator.rb
index 669565880e93..393563dead8a 100644
--- a/ee/lib/compliancemanagement/compliancerequirement/queryevaluator.rb
+++ b/ee/lib/compliancemanagement/compliancerequirement/queryevaluator.rb
@@ -85,6 +85,8 @@ def getfieldvalue(field)
          @project.visibility
        when 'removeapprovalswhennewcommitsareadded'
          @project.resetapprovalsonpush
```

+ when 'projectdisableoverridingapproverspermergerequest'
+ @project.disableoverridingapproverspermergerequest
when 'minimumapprovalsrequired'
@project.approvalrules.pick("SUM(approvalsrequired)") || 0
else

Constraints

Consideration	Constraint
Project applicability	Ultimate projects with associated Framework
Limit on frameworks per project	20
Limit on total requirements per framework	50
Limit on total controls per requirement	5
Limit on total fields per control expression	5
Allowed fields belonging to control expressions	Bounded allowlist tied to applicable schema
Control expressions query complexity	Files is non-user modifiable outside of configuration UI and subject to schema validation
Compliance validation frequency	12hrs

References

- https://gitlab.com/gitlab-org/software-supply-chain-security/compliance/general/-/issues/233
- https://handbook.gitlab.com/handbook/engineering/architecture/design-documents/compliance-adherence-reporting/

SECTION: engineering/architecture/design-documents/compliance-frameworks/decisions/001_triggering_checks.md

Context

We need to offer compliance tools to enable our customers to know if they adhere to certain rules, standards,
regulations, and guidelines. For this we need to create checks within GitLab that can be related to the software
development standards, frameworks, regulations or laws in the industry.

Approach

We thought of the following approaches for creating the concept of checks within GitLab:

1. Use Audit Events as Checks
1. Use Audit Events as trigger points for creating Checks
1. Use Sidekiq workers for creating and updating Checks (selected approach)

Example

Let us assume that we have a compliance check called "Prevent approval by merge request authors". To comply with this
check, we must prevent users from approving their own merge requests.

All the three approaches discussed below ensures that the status of the check is updated whenever the setting to
prevent users from approving their own merge request is modified.

Use Audit Events as Checks

In this approach we planned to directly use the existing Audit Events that are stored in the database as checks.

As an example, consider that Merge Request Approval Settings to prevent approval by merge request author is enabled.
This means that we have ensured that a merge request author cannot approve their own merge request.

On the compliance standards adherence dashboard, we planned to directly fetch the last audit event related to that
compliance check and would display the compliance status accordingly.

To further understand this, consider the example above, we would query the audit events table to get the
latest audit event that updated the merge request approval settings to prevent approval by merge request author.

Advantages

1. No need for creating new database tables for storing checks.
1. Audit events serving as the single source of truth for adherence to compliance standards.

Disadvantages

1. Currently, querying audit events table is already quite slow because of the large size of the database.
1. Not all the audit events are stored consistently, hence we would need to parse the JSON column details of the
auditevents table to get the details of the event.
1. Due to the above reasons, this approach would result in poor performance in terms of loading the compliance
standards dashboard.

Use Audit Events as trigger points for creating Checks

In this approach we planned to create a separate database table for storing the results of checks.

The status of these checks would be derived from the audit events via database triggers or Active Record callbacks.

To further understand this, consider the example above, whenever a new audit event is created, we would
check if that audit event is related to merge request approval settings being modified. We would create or update the
status of the check in the database accordingly.

Advantages

1. No need to query the audit events database table as the approach is event based.
1. Audit events serving as the single source of truth for adherence to compliance standards.
1. Increase code readability as the logic for creating checks